

Консольная игра «Балда»

1. Введение и назначение программы

Данная программа представляет собой компьютерную реализацию классической лингвистической настольной игры «Балда». Программа для 2 до 4 человек

Основная цель игры — поочередно добавляя по одной букве на игровое поле размером 5×5 , составлять новые существительные и набирать очки, равные количеству букв в составленном слове.

2. Объектная модель классов (Архитектура ПО)

Каждый класс представляет собой независимую сущность с приватными свойствами (переменными) и публичными (методами).

Диаграмма классов

 Объектная модель классов

Спецификация компонентов

1. Class Player (Сущность игрока)

- **Private свойства:**
 - `std::string name` — текстовое имя игрока (например, "Игрок 1").
 - `int score` — текущее количество набранных игроком очков.
- **Public методы:**
 - `Player(std::string n)` — конструктор для инициализации карточки игрока со счетом 0.
 - `getName() const` — безопасный геттер для чтения имени.
 - `getScore() const` — безопасный геттер для чтения текущего счёта.
 - `addPoints(int p)` — метод для начисления заработанных за ход очков.

2. Class GameBoard (Игровое поле)

Управляет состоянием сетки букв.

- **Private свойства:**
 - `std::string board[5][5]` — статическая двумерная матрица строк, полностью скрытая от прямого изменения извне.
- **Public методы:**
 - `GameBoard()` — конструктор, очищающий поле пробелами при старте.
 - `initCentralWord(const std::string& word)` — размещение стартового слова в центральной строке поля.
 - `display() const` — отрисовка сетки поля и текущих букв в консоли.

- `setLetter(int row, int col, const std::string& letter)` — установка символа (строки) в выбранную ячейку.
- `getLetter(int row, int col) const` — получение символа из конкретной ячейки.
- `isEmpty(int row, int col) const` — проверка, пуста ли выбранная клетка.
- `hasNeighbors(int row, int col) const` — проверка того, что новая буква соприкасается по граням с уже существующими буквами на поле.

3. Class GameDictionary (Словарь игры)

- **Private свойства:**
 - `std::set<std::string> valid_words` — база данных всех разрешенных слов, загруженная из файла `dictionary.txt`.
 - `std::set<std::string> used_words` — список слов, которые игроки уже составили в текущей сессии.
- **Public методы:**
 - `GameDictionary(const std::string& filename)` — конструктор, который парсит файл словаря и гарантированно очищает строки от скрытых символов переноса строк (`\r`, `\n`) для обеспечения точного посимвольного сравнения.
 - `isValid(const std::string& word) const` — проверка, существует ли придуманное слово в официальном словаре.
 - `isUsed(const std::string& word) const` — проверка, не называл ли кто-то это слово ранее в этой партии.
 - `addUsedWord(const std::string& word)` — занесение слова в список использованных (блокировка его для последующих ходов).

4. Class WordSearcher (Логический валидатор пути)

- **Private методы:**
 - `dfs(...)` — внутренний рекурсивный алгоритм поиска в глубину с механизмом отката.
- **Public методы:**
 - `canFormWord(const GameBoard& board, const std::string& word, int target_r, int target_c)` — внешний интерфейс, разбивающий строку на массив символов и запускающий сканирование поля «змейкой» для проверки введенного слова.

5. Class GameManager (Главный контроллер)

- **Private свойства:**
 - `GameBoard board` — объект игрового поля.
 - `GameDictionary dictionary` — объект словаря.
 - `WordSearcher searcher` — объект поисковика путей.
 - `std::vector<Player> players` — динамический список (вектор) объектов участников.
 - `int current_player_index` — индекс игрока, чей ход сейчас активен.
- **Public методы:**

- `GameManager(int players_count, const std::string& start_word, const std::string& dict_filename)` — конструктор, настраивающий компоненты игры и создающий участников.
- `start()` — запускает и поддерживает бесконечный игровой цикл ходов.

3. Алгоритмическая блок-схема (Логика хода)

Ниже представлена схема, описывающая алгоритм обработки одного хода игрока в методе `GameManager::start()`.

 Блок-схема логики хода

4. Описание ключевого алгоритма: Поиск в глубину (DFS)

Для проверки геометрической связности слова используется модифицированный алгоритм DFS с возвратом.

1. **Точка старта:** Алгоритм предварительно разбивает слово на массив букв и ищет ячейку поля, совпадающую с первой буквой.
2. **Рекурсивный обход:** Из найденной точки функция `dfs()` пытается сделать шаг в 4 направлениях: вверх, вниз, влево, вправо.
3. **Критерии отсеечения (База рекурсии):** Шаг считается ложным, если координаты вышли за пределы матрицы 5×5 , если строка в ячейке поля не совпадает с текущей буквой слова или если клетка уже была посещена в рамках текущего пути.
4. **Валидация новой буквы:** Важнейшее условие игры — слово должно включать новую букву, поставленную в этом раунде. Алгоритм проверяет, пересекались ли координаты пути с целевой координатой установленной буквы (`target_r`, `target_c`).
5. **Откат (Backtracking):** Если путь зашел в тупик, алгоритм снимает метку "посещено" с текущей клетки и пробует альтернативную ветку движения.

5. Техническая документация

5.1. Руководство программиста (Развертывание и компиляция)

- **Стандарт языка:** C++11 / C++17 / C++20.
- **Зависимости:** Проект использует стандартную библиотеку шаблонов (STL) и стандартные потоки ввода-вывода.
- **Файловая структура:** В корневой папке проекта обязательно должен находиться файл `dictionary.txt`.

Структура папок:

```
words/
├── main.cpp
├── GameManager.h / .cpp
└── GameBoard.h / .cpp
```

```
|— 📄 GameDictionary.h / .cpp
|— 📄 WordSearcher.h / .cpp
|— 📄 dictionary.txt      <-- Словник в кодировке UTF-8
|— 📁 words/
|   |— 📁 png/           <-- Папка для графических схем (qw, q, qwe)
**схема папок**
! [Иллюстрация алгоритма ] (words/png/qwe.png)
```